

Program No: 3

File Name:

Ex. No:

Date: _____

CREATE A PROGRESSIVE WEB APP (PWA) THAT CAN WORK OFFLINE

AIM:

To design and develop a Progressive Web App (PWA) using HTML, CSS, and JavaScript that can work offline by caching resources through a service worker.

ALGORITHM:

Step 1: Create an HTML file (index.html) as the main structure of the web app.

Step 2: Design the layout using a CSS stylesheet (style.css).

Step 3: Create a manifest.json file to describe the app's name, icons, and start URL for installability.

Step 4: Write a JavaScript file (service-worker.js) that defines caching logic for offline use.

Step 5: Register the service worker in app.js using the navigator.serviceWorker.register() method.

Step 6: Store required assets (HTML, CSS, JS, manifest, icons) inside a cache during the service worker's install event.

Step 7: Intercept network requests during the fetch event and serve cached files when offline.

Step 8: Build and organize all files in one project directory structure.

Step 9: Run a local server using the command `npx http-server` or `python -m http.server`.

Step 10: Open `http://localhost:8080` in a modern browser (Chrome/Edge).

Step 11: Verify offline functionality by turning on "Offline" mode in DevTools and refreshing the page.

Step 12: Confirm that the app loads from cache and remains accessible without network connectivity.

PROGRAM:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Offline PWA Example</title>

  <link rel="stylesheet" href="style.css" />
  <link rel="manifest" href="manifest.json" />
  <link rel="icon" href="icon-192.png" sizes="192x192" />
  <meta name="theme-color" content="#00796B" />
</head>
<body>
  <main>
    <h1>Offline Progressive Web App</h1>
    <p>This PWA works even when you're offline.</p>
  </main>

  <script src="app.js"></script>
</body>
</html>
```

```
html, body {
  margin: 0;
  padding: 0;
  height: 100%;
  background-color: #e0f2f1;
  color: #004d40;
  font-family: Arial, sans-serif;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

```
main {
  text-align: center;
}
```

```
{
  "name": "Offline PWA Example",
  "short_name": "OfflinePWA",
  "start_url": "./index.html",
```

```

"display": "standalone",
"background_color": "#e0f2f1",
"theme_color": "#00796B",
"icons": [
  {
    "src": "icon-192.png",
    "sizes": "192x192",
    "type": "image/png"
  },
  {
    "src": "icon-512.png",
    "sizes": "512x512",
    "type": "image/png"
  }
]
}
// Register the service worker
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker
      .register('service-worker.js')
      .then(reg => console.log('Service Worker registered:', reg))
      .catch(err => console.error('Service Worker registration failed:', err));
  });
}

const CACHE_NAME = 'pwa-offline-cache-v1';
const ASSETS = [
  '/',
  '/index.html',
  '/style.css',
  '/app.js',
  '/manifest.json',
  '/icon-192.png',
  '/icon-512.png'
];

// Install event – cache app shell
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => cache.addAll(ASSETS))
  );
  self.skipWaiting();
});

// Activate event – remove old caches

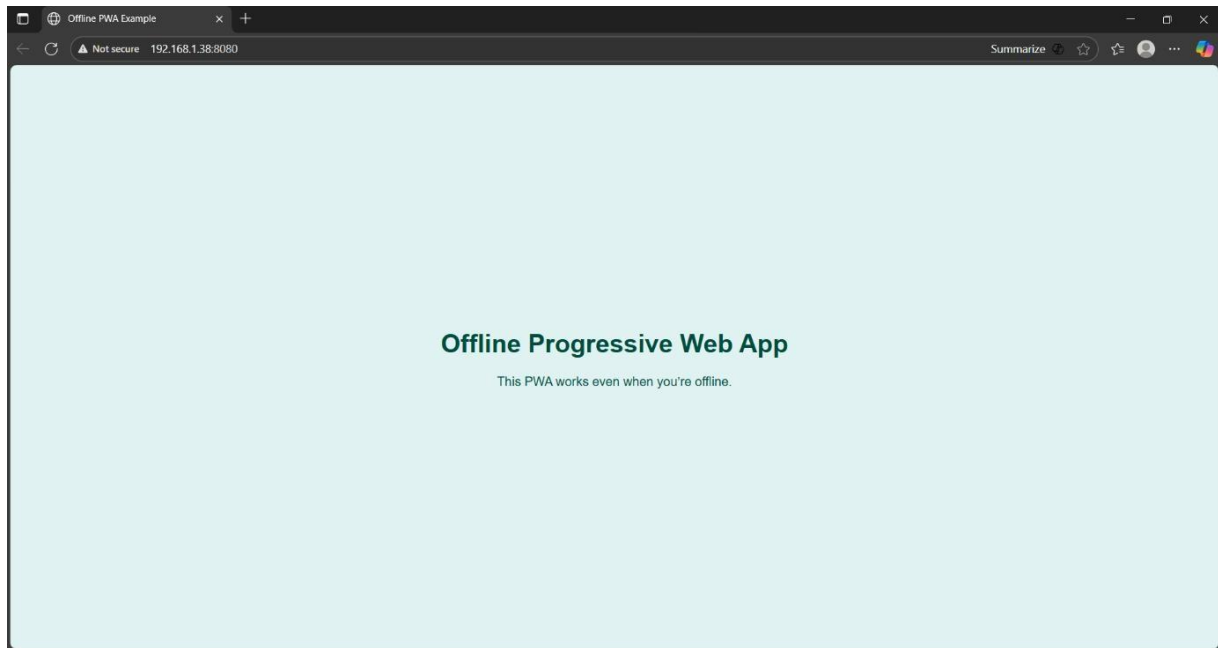
```

```
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(keys.filter(k => k !== CACHE_NAME).map(k => caches.delete(k)))
    )
  );
});
```

// Fetch event – serve cached resources when offline

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request)
      .then(resp => resp || fetch(event.request))
      .catch(() => caches.match('/index.html'))
  );
});
```

OUTPUT:



RESULT:

Thus the program was successfully executed.